

C Programming I

2024 Fall

Final

Instructor: Po-Wen Chi

Date: 2024.12.21 14:00 - 18:00

Policies:

- Online test.
- Do not forget to include your Makefile. TA will only use the command make to build your program. If make fails, you will get zero points and no room for bargaining. So if you do not know how to solve a problem, please, do not include it in your Makefile.
- I do not care your source code file names, but the executive binary names should be **fin01**, **fin02**, **fin03**, **fin04**.
- You can ask TA any questions on the moodle forum if you do not understand the problems. But remember, TA QA time is only 14:00-16:00.

1 Spherical Coordinate(25 pts)

Do you know what Spherical Coordinate is? Fig. 1 is an example. If you need more information, please reference the following link.

<https://zh.wikipedia.org/zh-tw/%E7%90%83%E5%BA%A7%E6%A8%99%E7%B3%BB>

https://en.wikipedia.org/wiki/Spherical_coordinate_system

Given a point in a Cartesian coordinate system, I want you to develop a series of functions to move this point in r, θ, ϕ .

```
1 // Move the given point according to the function.
2 // Input:
3 // double *pX, *pY, *pZ: the given point position.
4 // double r, theta, phi: change the original value of the point according to
  the input argument.
5 // That is,
6 // new radius distance = old radius distance + r
7 // new polar angle = old polar angle + theta
8 // new azimuth angle = old azimuth angle + phi
9 // Output:
10 // double *pX, *pY, *pZ: the new point position.
```

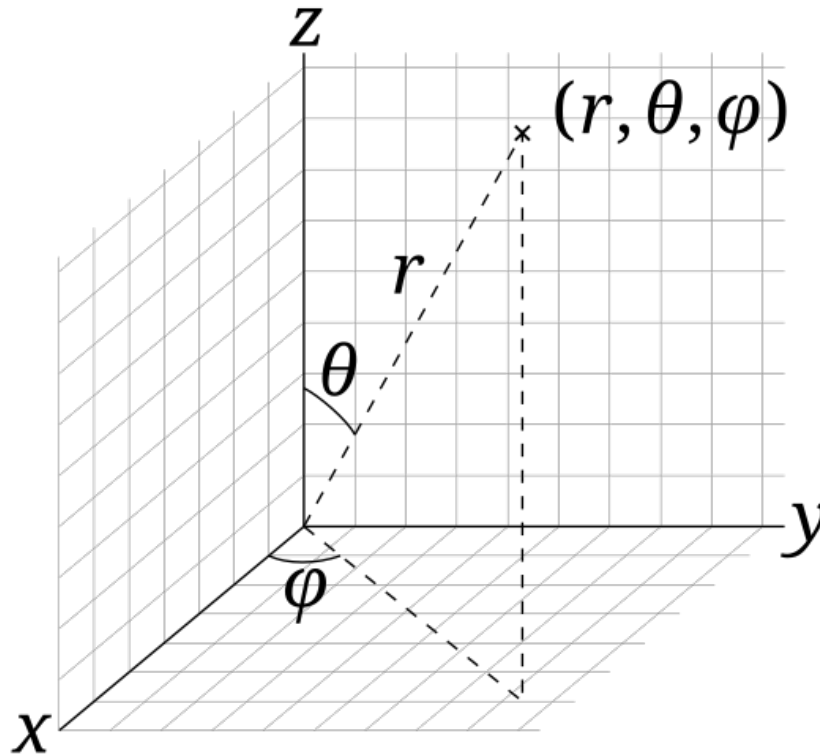


Figure 1: Spherical Coordinate.

```

11 // Return:
12 // For any invalid input or radius distance is less than zero, return -1;
   otherwise, return 0.
13 int32_t changeR( double *pX, double *pY, double *pZ, double r );
14 int32_t changeTheta( double *pX, double *pY, double *pZ, double theta );
15 int32_t changePhi( double *pX, double *pY, double *pZ, double phi );

```

You should prepare a header file called `spherical.h`. Our TAs will prepare `fin01.c` which includes `spherical.h` and uses this function. Do not forget to make `fin01.c` to `fin01` in your Makefile.

2 T-score Curve (25 pts)

I know you all want to get high GPA, right? As you can see, some courses have many classes, where there are many different teachers for one course. Some teachers prefer giving high scores while others may prefer giving low scores. Is this fair? Of course NOT. To solve this problem, NTHU (清華大學) uses T-score to balance the difference between classes. I will show you how to do T-score curve. Given a set of scores $\{X_i\}$, the curved score X'_i will be

$$X'_i = A + B \cdot \frac{X_i - \bar{X}}{\sigma}.$$

Here A is the preferred average score after curving, B is the preferred standard deviation

after curving, $\bar{X}$ is the average score before curving and σ is the standard deviation before curving.

If you are not familiar with T-score, maybe you can reference the following document:

<https://registra.site.nthu.edu.tw/var/file/211/1211/img/4247/131314238.pdf>

In the NTHU document, the average score A is 50 and the preferred standard deviation B is 10.

In this problem, I want you to develop a library to record students' scores in many classes. All scores should be **double**.

```
1 // Setup the class number of this course.
2 // Note that the class ID is from 1 to classNumber.
3 // Of course, you need to allocate memory in this function.
4 // For your simplicity, in this function, you do not need to consider the case
   that the memory is not enough.
5 void setupCourse( double ***, uint32_t classNumber );
6
7 // Setup the student number of a class where the class ID is classId.
8 // Note that the student ID is from 1 to studentNumber.
9 // Of course, you need to allocate memory in this function.
10 // For your simplicity, in this function, you do not need to consider the case
   that the memory is not enough. Besides, I promise that the input classId
   is valid.
11 void setupClass( double **, uint32_t classId, uint32_t studentNumber );
12
13 // Input the score of a given student whose class ID is classId and student ID
   is studentId.
14 // For your simplicity, I promise classId and studentId are valid.
15 // If the input pointer is NULL, return -1; otherwise, return 0.
16 int32_t inputScore( double **, uint32_t classId, uint32_t studentId, double
   score );
17
18 // Get the best/worst score of all students.
19 // If the input pointer is NULL, return -1; otherwise, return 0.
20 int32_t findBest( double **, double *pScore );
21 int32_t findWorst( double **, double *pScore );
22
23 // Curve all scores with T-score approaches.
24 // If the input pointer is NULL, return -1; otherwise, return 0.
25 int32_t tscore( double **, double preferredAverage, double preferredStdDev );
26
27 // Free All allocated memory.
28 void freeCourse( double ** );
```

For your reference, an example library usage is as follows.

```
1
2 double **ppScores = NULL;
3
4 setupCourse( &ppScores, 10 ); // 總共有 10 個班級
5 setupClass( ppScores, 1, 50 ); // 第一個班級有 50 個學生
6 inputScore( ppScores, 1, 2, 90 ); // 第一個班級的 2 號學生的成績是 90 分
7 ...
8 double best = 0;
9 double worst = 0;
```

```

10
11 findBest( ppScores, &best );    // 最佳成績
12 findWorst( ppScores, &worst );  // 最差成績
13 tscore( ppScores, 50, 10 );    // T分數調整
14 findBest( ppScores, &best );    // 最佳成績
15 findWorst( ppScores, &worst );  // 最差成績
16
17 freeCourse( ppScores );

```

If a student's score is not input, treat the score as **zero**. You should also prepare a header file called **myscore.h**. Our TAs will prepare **fin02.c** which includes **myscore.h** and uses this function. Do not forget to make **fin02.c** to **fin02** in your Makefile.

3 My Memory Debug Tool (25 pts)

In this class, I have told you some horrible stories about **memory leak**. To solve the memory leak issue, I want you to develop a memory management library for memory debugging.

```

1 // Allocate memory for the given size.
2 void *myMalloc( size_t size );
3
4 // Free the allocated memory.
5 // Note that this is only used for the memory allocated through myMalloc.
6 void myFree( void *ptr );
7
8 // Get the total allocated memory size through myMalloc.
9 size_t getAllocatedMemSize( void );
10
11 // Get the total allocated memory size through myMalloc in the time period
    between timeA and timeB.
12 size_t getAllocatedMemSizePeriod( time_t timeA, time_t timeB );
13
14 // Print all allocated memory blocks through myMalloc.
15 // Print One Block format:
16 // ==
17 // Time: 1733385903, Size: 32
18 // 0x55c8003736b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
19 // 0x55c8003736c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
20 // ==
21 // The blocks should be ordered according to the time sequence.
22 // You should print addresses and data of the memory block, where each row
    contains 16 bytes.
23 // Return: the total allocated memory size through myMalloc.
24 size_t dumpAllocatedMemSize( void );
25
26 // Print all allocated memory block through myMalloc in the time period
    between timeA and timeB.
27 // Print format is the same with dumpAllocatedMemSize.
28 // Return: the total allocated memory size through myMalloc in the time period
    between timeA and timeB.
29 size_t dumpAllocatedMemSizePeriod( time_t timeA, time_t timeB );

```

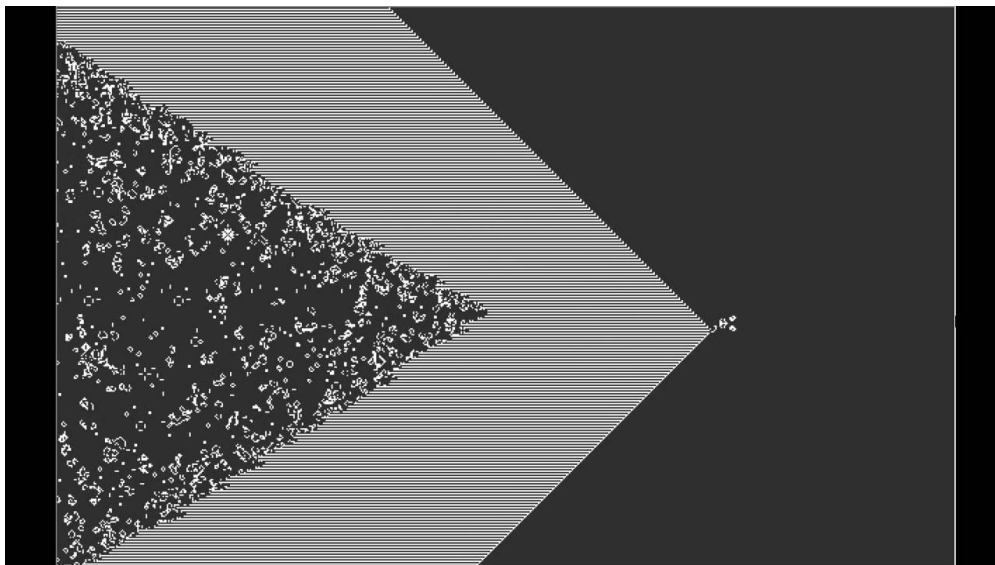


Figure 2: Conway's Game of Life

Note that once a memory block is freed, the size should not be counted and the content should not be dumped. You should also prepare a header file called `mymem.h`. Our TAs will prepare `fin03.c` which includes `mymem.h` and uses this function. Do not forget to make `fin03.c` to `fin03` in your Makefile.

4 NO GAME NO LIFE (25 pts)

The Game of Life, also known as Conway's Game of Life or simply Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970.

It is a zero-player game, meaning that its evolution is determined by its initial state, requiring no further input.

One interacts with the Game of Life by creating an initial configuration and observing how it evolves. It is Turing complete and can simulate a universal constructor or any other Turing machine.

For more information, refer to the Wikipedia page.

Rules

The universe of the Game of Life is a two-dimensional orthogonal grid of square cells, each of which is in one of two possible states: live or dead (populated or unpopulated, respectively).

Each cell interacts with its eight neighbors, which are the cells that are horizontally, vertically, or diagonally adjacent. At each step in time, the following transitions occur:

1. Any live cell with fewer than 2 live neighbors dies, as if by underpopulation.
2. Any live cell with 2 or 3 live neighbors lives on to the next generation.

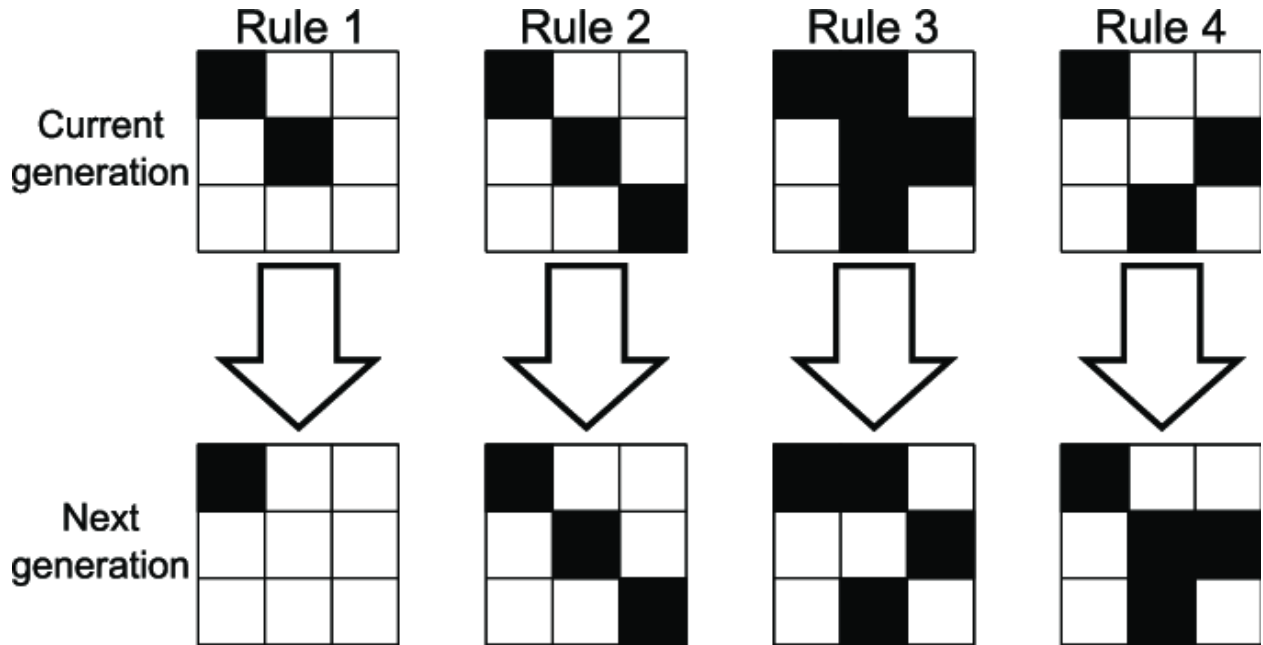


Figure 3: Rule about the center cells.

3. Any live cell with more than 3 live neighbors dies, as if by overpopulation.
4. Any dead cell with exactly 3 live neighbors becomes a live cell, as if by reproduction.

The first generation is created by applying the above rules simultaneously to every cell in the seed, live or dead; births and deaths occur simultaneously, and the discrete moment at which this happens is sometimes called a "tick." Each generation is a pure function of the preceding one. The rules continue to be applied repeatedly to create further generations.

For your convenience, i suggest you play the online version below, even the english wiki page has a game, [emacs](#) has a build-in game too.

BTW, if you search for "[Conway's Game of Life](#)" on google, there is an Easter egg.

- [english Wiki page](#).
- [playgameoflife.com](#)
- [academo.org](#)

The third version has handle edges with cycle boundary, which fit our requirement below, please see specifications.

Specifications

The initial pattern constitutes the seed of the system. there is 6 MACRO in [testdata.h](#), those data may change but not exceed 256.

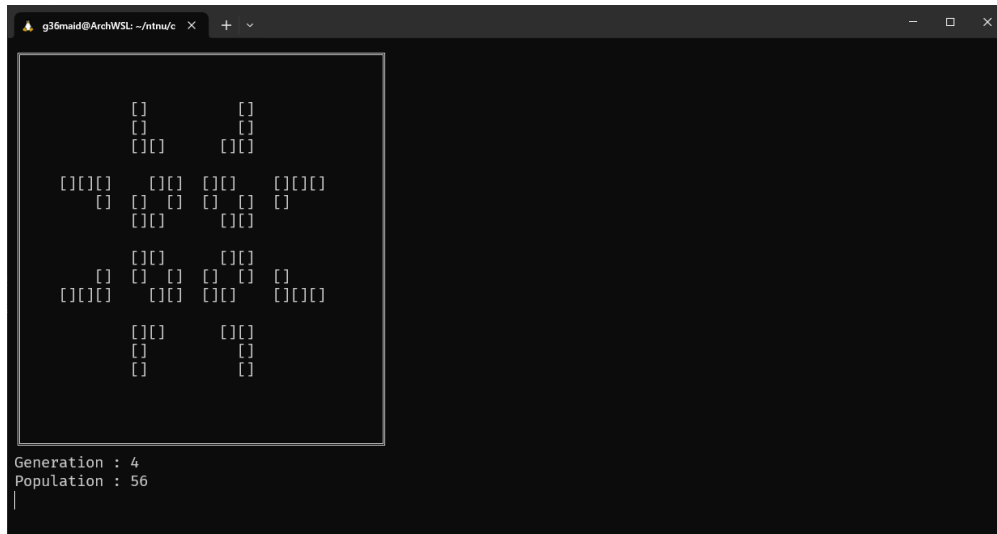


Figure 4: Example Pattern: Pulsar

```

1 #define ROWS 20
2 #define COLS 20
3 #define INPUT_ROWS 15
4 #define INPUT_COLS 15
5 #define POSITION_ROW 2 //position is 0-based
6 #define POSITION_COL 2

```

Simulating an infinite grid is very hard. Therefore, we will simulate a grid with a fixed size.

To handle edges, the left and right edges of the grid are considered to be "stitched" together, and the top and bottom edges as well, yielding a circular 2D array (Like the game "PACMAN").

That means that the edge cells will be affected by the cell on the other side. (You can run the Spaceship or Glider at academo.org to see what happen.)

The grid size, examples of patterns, and initial positions of patterns are defined in `test-data.h`. You should include the file, which is available on Google Cloud [here](#).

Similar to Homework 2, We use box-drawing characters to draw the edge of the board. Each live cell should be displayed as "[]" inside the board.

Additionally, print the **generation** number and the total **population** for each round.

Refresh the board every second and clear the screen before redrawing using the following code:

```

1 printf("\033[2J"); // clear screen
2 //print your game now
3 sleep(1); // or usleep(1000000) to sleep 1 second
4 printf("\033[H"); // reset the cursor top left

```

Notes

- Ensure that your program adheres to the rules of the Game of Life.

- - Use the provided `testdata.h` file for initial configurations.
- - Your implementation should handle a toroidal grid correctly.
- - The program should continuously display the board and update it every second.
- - Clearly show the generation number and population for each iteration.

Box-drawing characters

Wikipedia page.

- (U+2550) Box Drawings Light Horizontal
- (U+2551) Box Drawings Light Vertical
- (U+2554) Box Drawings Light Down and Right
- (U+2557) Box Drawings Light Down and Left
- (U+255A) Box Drawings Light Up and Right
- (U+255D) Box Drawings Light Up and Left

5 Bonus: Your Comments (5 pts)

Again, any comments are welcomed. However, you will get nothing if you leave this question blank.

Bonus: GoogleTest

Google Test, often referred to as **gtest**, is a specialized library utilized to conduct unit testing in the C++ programming language. Undoubtedly, it can also be used in C. The project site is as follows.

<https://google.github.io/googletest/>

Please write a tutorial about **gtest** for additional **3 points** in this class.

Due : 12/26 23 : 59

Bonus: 問卷調查

因為我實驗室有個碩士生在研究上需要收集問卷，想麻煩各位幫忙填寫一下。為了感謝各位的幫忙，填寫問卷可以有**總成績兩分**的加分。另外，雖然問卷有收集各位的姓名學號，但那純粹是為了加分使用，不會列入研究。

<https://forms.gle/TWbxboVpvMCUN2hi7>

截止日期：12/22 23 : 59

Advertisement: Software Engineering Final Presentation

Welcome to join the final presentation of the class, Software Engineering.

- Date: 2024/12/27
- Time: AM 9:00
- Location: S102